

Московский Авиационный Институт
(Национальный Исследовательский Университет)
Факультет информационных технологий и прикладной математики
Кафедра вычислительной математики и программирования

**Лабораторная работа №5-7 по курсу
«Операционные системы»**

**Тема работы
“Брокеры сообщений”**

Студент: Салихов Руслан Ринатович
Группа: М8О-203Б-23
Вариант: 48

Преподаватель: Миронов Евгений Сергеевич

Оценка: _____
Дата: _____
Подпись: _____

Москва, 2024

Постановка задачи

Задача: Реализовать распределённую систему по асинхронной обработке запросов, в которой существуют два вида узлов – управляющий и вычислительный. Необходимо объединить данные узлы в идеально сбалансированное бинарное дерево, где каждый следующий узел добавляется в самое наименьшее поддереву. Связь между узлами должна осуществляться с использованием технологии очередей сообщений (ZeroMQ). Также система должна предусматривать проверку доступности узлов с помощью механизма heartbeat – если от вычислительного узла в течение 4×установленного интервала heartbeat не поступают сообщения, узел считается недоступным, однако родительские узлы продолжают функционировать. Управляющий узел принимает команды от пользователя и маршрутизирует их на вычислительные узлы, которые реализуют функциональность локального целочисленного словаря (сохранение и извлечение числовых значений по строковому ключу).

Общие сведения о программе

Программа реализует управление процессами и их взаимодействие через дерево узлов, организованное в виде идеально сбалансированного бинарного дерева. Основным элементом реализации является класс Node, который инкапсулирует логику создания дочерних узлов, обмена сообщениями посредством ZeroMQ и работу с локальным словарём для команд exes. Управляющий узел (с идентификатором –1) инициирует создание новых вычислительных узлов, передаёт им команды и следит за их доступностью через механизм heartbeat. Для обмена сообщениями используется схема REQ/REP, что гарантирует корректное завершение каждого цикла передачи сообщений. При выполнении команды exes вычислительные узлы либо сохраняют числовое значение по заданному ключу, либо возвращают сохранённое значение, если параметр не передан. Команда ring позволяет проверить доступность конкретного узла, а команда heartbit задаёт интервал, по которому узлы периодически отправляют сообщения о своей работоспособности. Дополнительно реализован поток проверки heartbeat,

который отслеживает время последнего полученного сообщения от каждого дочернего узла и информирует пользователя о недоступных узлах.

Выводы

В ходе выполнения работы была разработана распределённая система для асинхронной обработки запросов, в которой узлы объединены в идеально сбалансированное бинарное дерево. Применение технологии ZeroMQ обеспечило надёжное межпроцессное взаимодействие, а реализация механизма heartbeat позволила эффективно мониторить состояние вычислительных узлов. Использование локального целочисленного словаря в вычислительных узлах продемонстрировало возможность хранения и извлечения данных по ключу, что является важным аспектом распределённых вычислений. Полученная система демонстрирует высокую гибкость, масштабируемость и отказоустойчивость, подтверждая практическую применимость методов системного программирования и асинхронного обмена сообщениями для реализации распределённых вычислительных систем.

Код лаб. Работы:

Src/client.cpp

```
#include <set>
#include <iostream>
#include <sstream>
#include <memory>
#include "../include/tree.hpp"
#include "../include/manage_zmq.hpp"
#include <cstdlib>

// export PROGRAM_PATH="/mnt/c/Users/user/Desktop/Labs/os/lr5-2/build/server"

int main() {
    std::set<int> Nodes;
    char* programPathC = getenv("PROGRAM_PATH");
```

```

if (!programPathC) {
    std::cerr << "Error: PROGRAM_PATH environment variable not set" << std::endl;
    return 1;
}
std::string programPath = programPathC;
Node task(-1);
Nodes.insert(-1);
std::string command;
while (std::cin >> command) {
    if (command == "create") {
        int idChild, idParent;
        std::cin >> idChild >> idParent;
        if (Nodes.find(idChild) != Nodes.end()) {
            std::cout << "Error: Already exists" << std::endl;
        } else if (Nodes.find(idParent) == Nodes.end()) {
            std::cout << "Error: Parent not found" << std::endl;
        } else if (idParent == task.id) { // от управляющего узла
            std::string ans = task.Create(idChild, programPath);
            std::cout << ans << std::endl;
            Nodes.insert(idChild);
        } else { // пересылаем команду родительскому узлу
            std::string str = "create " + std::to_string(idChild);
            std::string ans = task.Send(str, idParent);
            std::cout << ans << std::endl;
            Nodes.insert(idChild);
        }
    } else if (command == "heartbit") {
        int newInterval;
        std::cin >> newInterval;
        task.SetHeartbeatInterval(newInterval);
        std::cout << "Ok" << std::endl;
    } else if (command == "ping") {
        int idChild;
        std::cin >> idChild;
    }
}

```

```

if (Nodes.find(idChild) == Nodes.end()) {
    std::cout << "Error: Not found" << std::endl;
} else if (task.children.find(idChild) != task.children.end()) {
    std::string ans = task.Ping(idChild);
    std::cout << ans << std::endl;
} else {
    std::string str = "ping " + std::to_string(idChild);
    std::string ans = task.Send(str, idChild);
    if (ans == "Error: Not found") {
        ans = "Ok:0";
    }
    std::cout << ans << std::endl;
}
} else if (command == "exec") {
    int id;
    std::cin >> id;
    if (Nodes.find(id) == Nodes.end()) {
        std::cout << "Error: Not found" << std::endl;
        continue;
    }
    std::string name;
    std::cin >> name;
    std::string remainder;
    std::getline(std::cin, remainder);
    std::istringstream iss(remainder);
    int value;
    if (iss >> value) { // сохранение значения
        std::string msg = "exec " + std::to_string(id) + " " + name + " " + std::to_string(value);
        std::string ans = task.Send(msg, id);
        std::cout << ans << std::endl;
    } else { // загрузка значения
        std::string msg = "exec " + std::to_string(id) + " " + name;
        std::string ans = task.Send(msg, id);
        std::cout << ans << std::endl;
    }
}

```

```

    }

    } else if (command == "kill") {
        int id;
        std::cin >> id;
        std::string msg = "kill";
        if (Nodes.find(id) == Nodes.end()) {
            std::cout << "Error: Not found" << std::endl;
        } else {
            std::string ans = task.Send(msg, id);
            if (ans != "Error: Not found") {
                std::istringstream ids(ans);
                int tmp;
                while (ids >> tmp) {
                    Nodes.erase(tmp);
                }
                ans = "Ok";
                if (task.children.find(id) != task.children.end()) {
                    Unbind(task.children[id].get(), task.childrenPort[id]);
                    task.children[id]->close();
                    task.children.erase(id);
                    task.childrenPort.erase(id);
                }
            }
            std::cout << ans << std::endl;
        }

    } else if (command == "exit") {
        task.Kill();
        return 0;
    }

}

return 0;
}

```

Src/manage_zmq.cpp

```
#include "../include/manage_zmq.hpp"
#include <cstring>

int Bind(zmq::socket_t *socket, int id) {
    int port = 4040 + id;
    while (true) {
        std::string address = "tcp://127.0.0.1:" + std::to_string(port);
        try {
            socket->bind(address);
            break;
        } catch (...) {
            port++;
        }
    }
    return port;
}

void Unbind(zmq::socket_t *socket, int port) {
    std::string address = "tcp://127.0.0.1:" + std::to_string(port);
    socket->unbind(address);
}

void Connect(zmq::socket_t *socket, int port) {
    std::string address = "tcp://127.0.0.1:" + std::to_string(port);
    socket->connect(address);
}

void Disconnect(zmq::socket_t *socket, int port) {
    std::string address = "tcp://127.0.0.1:" + std::to_string(port);
    socket->disconnect(address);
}

bool SendMessage(zmq::socket_t *socket, const std::string& msg) {
```

```

    if (!socket || !(*socket)) {
        return false;
    }

    zmq::message_t message(msg.size());
    memcpy(message.data(), msg.c_str(), msg.size());
    try {
        socket->send(message, zmq::send_flags::none);
        return true;
    } catch (const zmq::error_t &e) {
        std::cerr << "SendMessage error: " << e.what() << std::endl;
        return false;
    }
}

std::optional<std::string> ReceiveMessage(zmq::socket_t* socket) {
    zmq::message_t message;
    try {
        auto result = socket->recv(message, zmq::recv_flags::none);
        if (result && *result > 0) {
            return std::string(static_cast<char*>(message.data()), message.size());
        }
    } catch (const zmq::error_t &e) {
        std::cerr << "ReceiveMessage warning: " << e.what() << std::endl;
    }
    return std::nullopt;
}

```

Src/server.cpp

```

#include "../include/tree.hpp"
#include "../include/manage_zmq.hpp"
#include <fstream>
#include <signal.h>
#include <map>
#include <thread>

```



```

#include <sstream>
#include <cstdlib>

int main(int argc, char **argv) {
    if (argc != 3) {
        perror("Not enough arguments");
        exit(EXIT_FAILURE);
    }
    int nodeId = atoi(argv[1]);
    int parentPort = atoi(argv[2]);
    Node task(nodeId, parentPort);
    Bind(&task.parent, nodeId);
    char* programPathC = getenv("PROGRAM_PATH");
    std::string programPath = (programPathC ? programPathC : "");

    while (true) {
        std::string message;
        if (auto msg = ReceiveMessage(&(task.parent)); msg.has_value()) {
            message = *msg;
        } else {
            continue;
        }
        std::istringstream request(message);
        std::string command;
        request >> command;
        if (command == "create") {
            int idChild;
            request >> idChild;
            std::string ans = task.Create(idChild, programPath);
            SendMessage(&task.parent, ans);
        } else if (command == "pid") {
            std::string ans = task.Pid();
            SendMessage(&task.parent, ans);
        } else if (command == "ping") {

```

```

    int idChild;
    request >> idChild;
    std::string ans = task.Ping(idChild);
    SendMessage(&task.parent, ans);
} else if (command == "send") {
    int id;
    request >> id;
    std::string str;
    getline(request, str);
    if (!str.empty() && str[0] == ' ') {
        str.erase(0, 1);
    }
    std::string ans = task.Send(str, id);
    SendMessage(&task.parent, ans);
} else if (command == "exec") {
    int targetId;
    request >> targetId;
    std::string name;
    request >> name;
    std::string remainder;
    std::getline(request, remainder);
    std::istringstream iss(remainder);
    int value;
    if (iss >> value) { // команда сохранения
        task.setKeyValue(name, value);
        std::string to_send = "Ok:" + std::to_string(targetId);
        SendMessage(&task.parent, to_send);
    } else { // команда загрузки
        auto valueOpt = task.getKeyValue(name);
        if (valueOpt.has_value()) {
            std::string to_send = "Ok:" + std::to_string(targetId) + ": " +
std::to_string(valueOpt.value());
            SendMessage(&task.parent, to_send);
        } else {
            std::string to_send = "Ok:" + std::to_string(targetId) + ": " + name + " not found";

```

```

        SendMessage(&task.parent, to_send);
    }
}
} else if (command == "heartbeat") {
    int senderId;
    request >> senderId; // Используем 'request', а не 'iss'
    task.updateHeartbeat(senderId);
    SendMessage(&task.parent, "Ok");
} else if (command == "kill") {
    std::string ans = task.Kill();
    ans = std::to_string(task.id) + " " + ans;
    SendMessage(&task.parent, ans);
    Disconnect(&task.parent, task.parentPort);
    task.parent.close();
    break;
}
}
return 0;
}

```

Src/tree.cpp

```

#include <unistd.h>
#include <sys/types.h>
#include <sstream>
#include "tree.hpp"
#include <thread>
#include <chrono>

std::string Node::Ping(int _id) {
    std::string ans = "Ok:0";
    if (_id == id) {

```

```

    ans = "Ok:1";
    return ans;
} else if (auto it = children.find(_id); it != children.end()) {
    std::string msg = "ping " + std::to_string(_id);
    {
        std::lock_guard<std::mutex> lock(childrenSocketMutex);
        SendMessage(it->second.get(), msg);
    }
    {
        std::lock_guard<std::mutex> lock(childrenSocketMutex);
        if (auto msg_resp = ReceiveMessage(children[_id].get());
            msg_resp.has_value() && *msg_resp == "Ok:1") {
            ans = *msg_resp;
        }
    }
    return ans;
}
return ans;
}

std::string Node::Create(int idChild, const std::string& programPath) {
    int newPort = 4040 + idChild;
    int pid = fork();

    if (pid == 0) {
        std::this_thread::sleep_for(std::chrono::milliseconds(300));
        execl(programPath.c_str(),
              programPath.c_str(),
              std::to_string(idChild).c_str(),
              std::to_string(newPort).c_str(),
              nullptr);
        _exit(1);
    } else {
        std::this_thread::sleep_for(std::chrono::milliseconds(500));
    }
}

```

```

    auto childSocket = std::make_unique<zmq::socket_t>(context, ZMQ_REQ);
    Connect(childSocket.get(), newPort);
    children[idChild] = std::move(childSocket);
    childrenPort[idChild] = newPort;
    children[idChild]->set(zmq::sockopt::sndtimeo, 3000);
    {
        std::lock_guard<std::mutex> lock(childrenSocketMutex);
        if (!SendMessage(children[idChild].get(), "pid")) {
            return "Error: failed to send pid request";
        }
    }
    std::string pidChild = "Error: couldn't connect to child";
    {
        std::lock_guard<std::mutex> lock(childrenSocketMutex);
        if (auto msg = ReceiveMessage(children[idChild].get()); msg.has_value()) {
            pidChild = *msg;
        }
    }
    return "Ok:" + pidChild;
}

std::string Node::Pid() {
    return std::to_string(getpid());
}

std::string Node::Send(const std::string& str, int id) {
    if (children.empty()) {
        return "Error: Not found";
    } else if (auto it = children.find(id); it != children.end()) {
        {
            std::lock_guard<std::mutex> lock(childrenSocketMutex);
            if (SendMessage(it->second.get(), str)) {
                std::string ans = "Error: Not found";
            }
        }
    }
}

```

```

        if (auto msg = ReceiveMessage(children[id].get()); msg.has_value()) {
            ans = *msg;
        }
        return ans;
    }
}
} else {
    std::string ans = "Error: Not found";
    for (auto& child : children) {
        std::string msg = "send " + std::to_string(id) + " " + str;
        {
            std::lock_guard<std::mutex> lock(childrenSocketMutex);
            if (SendMessage(child.second.get(), msg)) {
                if (auto msg_resp = ReceiveMessage(child.second.get()); msg_resp.has_value()) {
                    ans = *msg_resp;
                }
            }
        }
    }
    return ans;
}
return "Error: Not found";
}

std::string Node::Kill() {
    std::string ans;
    for (auto& child : children) {
        std::string msg = "kill";
        {
            std::lock_guard<std::mutex> lock(childrenSocketMutex);
            if (SendMessage(child.second.get(), msg)) {
                if (auto tmp = ReceiveMessage(child.second.get()); tmp.has_value()) {
                    msg = *tmp;
                }
            }
        }
    }
}

```

```

    }

    }

    Unbind(child.second.get(), childrenPort[child.first]);
    child.second->close();
    if (!ans.empty()) {
        ans += " " + msg;
    } else {
        ans = msg;
    }
}

children.clear();
childrenPort.clear();
return ans;
}

void Node::HeartbeatSender() {
    while (running) {
        std::this_thread::sleep_for(std::chrono::milliseconds(heartbeatInterval));

        std::string heartbeatMsg = "heartbeat " + std::to_string(id);
        // Отправляем heartbeat каждому дочернему узлу
        for (auto& child : children) {
            {
                std::lock_guard<std::mutex> lock(childrenSocketMutex);
                if (SendMessage(child.second.get(), heartbeatMsg)) {
                    // Сразу после отправки ждем ответ, чтобы завершить цикл REQ/REP
                    auto reply = ReceiveMessage(child.second.get());
                    if (!reply.has_value()) {
                        std::cerr << "Heartbeat: no reply from child " << std::endl;
                    }
                }
            }
        }
    }
}
}

```

```

void Node::HeartbeatReceiver() {
    while (running) {
        std::string message;
        {
            std::lock_guard<std::mutex> lock(parentSocketMutex);
            if (auto msg = ReceiveMessage(&parent); msg.has_value()) {
                message = *msg;
            } else {
                std::this_thread::sleep_for(std::chrono::milliseconds(100));
                continue;
            }
        }
        std::istringstream iss(message);
        std::string cmd;
        iss >> cmd;
        if (cmd == "heartbeat") {
            int senderId;
            iss >> senderId;
        }
        std::this_thread::sleep_for(std::chrono::milliseconds(100));
    }
}

void Node::setKeyValue(const std::string& key, int value) {
    std::lock_guard<std::mutex> lock(mtx);
    keyValueStore[key] = value;
}

std::optional<int> Node::getKeyValue(const std::string& key) const {
    std::lock_guard<std::mutex> lock(mtx);
    auto it = keyValueStore.find(key);
    if (it != keyValueStore.end()) {
        return it->second;
    }
}

```



```
}  
    return std::nullopt;  
}
```

Пример вывода:

```
user@DESKTOP-Q8EOMG:/mnt/c/Users/user/Desktop/Labs/os/lr5_com/build$ ./client  
create 1 -1  
Ok:680  
ping 1  
Ok:1  
create 2 1  
Ok:773  
exec 2 var 1  
Ok:2  
exec 2 var 3  
Ok:2  
exec 2 bar 4  
Ok:2  
exec 2 var  
Ok:2: 3  
heartbit 2000  
Ok  
ping 1  
Ok:1  
ping 2  
Ok:1  
exit  
user@DESKTOP-Q8EOMG:/mnt/c/Users/user/Desktop/Labs/os/lr5_com/build$
```